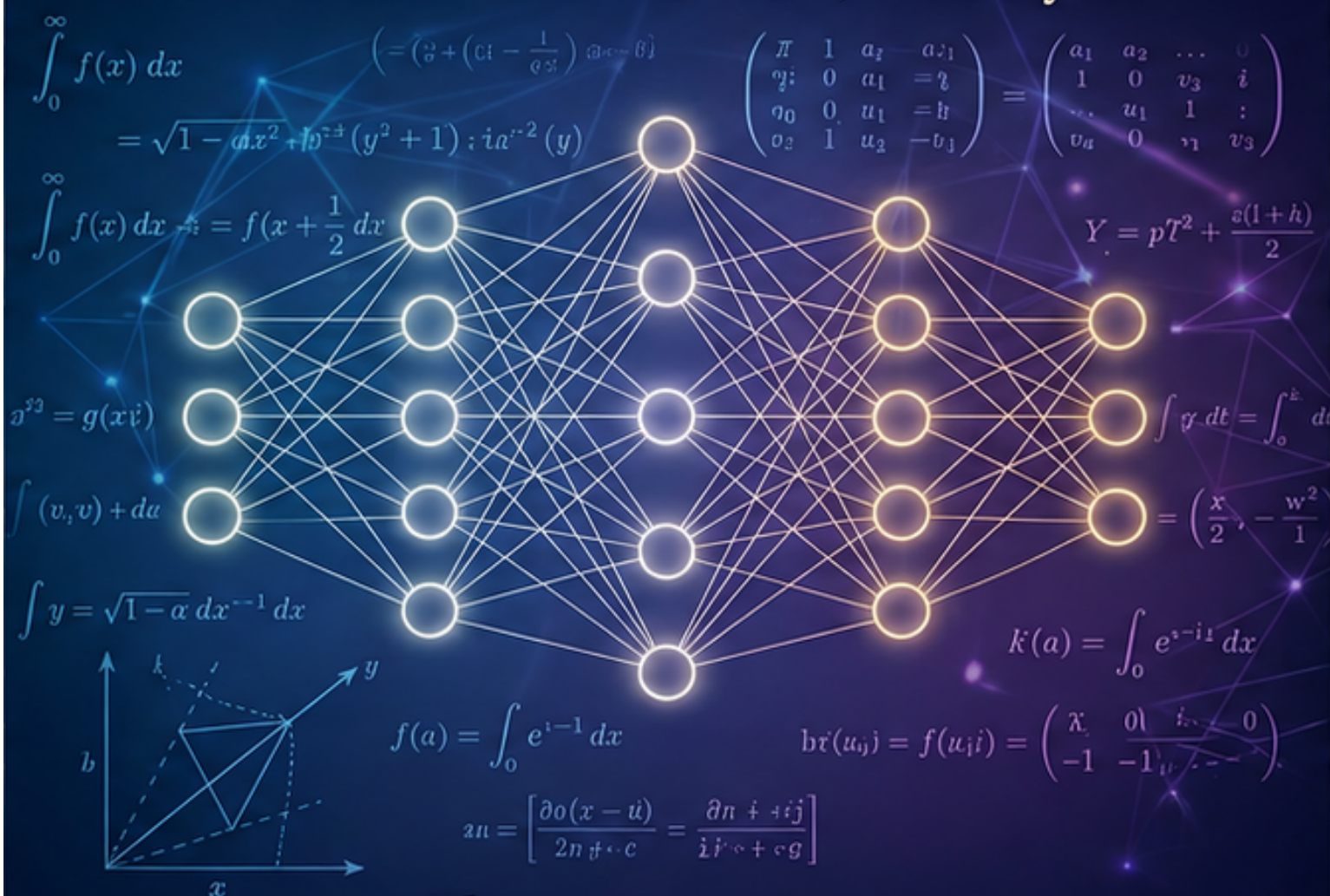


The Mathematical Intuition Behind Deep Learning

From the Dot Product to Multivariate Calculus and the Jacobian, with Python



Alex Punnen

The Mathematical Intuition Behind Deep Learning

From the Dot Product to Multivariate Calculus and the Jacobian, with Python

Alex Punnen

© All Rights Reserved

Contents

- Chapter 1: [The simplest Neural Network - Perceptron using Vectors and Dot Products](#)
 - Chapter 2: [Perceptron Training via Feature Vectors & HyperPlane split](#)
 - Chapter 3: [Gradient Descent and Optimization](#)
 - Chapter 4: [Backpropagation with Scalar Calculus](#)
 - Chapter 5: [Backpropagation with Matrix Calculus](#)
 - Chapter 6: [Backpropagation with Softmax and Cross Entropy](#)
 - Chapter 7: [Neural Network Implementation](#)
-

Notation

Symbols are defined in each chapter as they are introduced; a few conventions hold throughout:

- **Superscript = layer, subscript = component.** a^2 is the activation of layer 2; p_i is the i -th element of vector p .
- **Column vectors.** A layer computes $z^l = W^l a^{l-1} + b^l$, and gradients are column vectors, so an error is pushed back through a layer as $(W^l)^T \delta^l$.
- C is the cost (loss) and η is the learning rate.
- Chapter 7's NumPy code uses the row-vector / batch convention ($x @ w$); the note at the start of that chapter explains the transposition.

The simplest Neural Network - Perceptron using Vectors and Dot Products

Even the most complex Neural network is based on vectors and matrices, and it uses the concept of a cost function and algorithms like gradient descent to find a reduced cost. Then, it propagates the cost back to all constituents of the network proportionally via a method called back-propagation.

Have you ever held an integrated circuit or chip in your hand or seen one? It looks overwhelmingly complex. But its base is the humble transistor and Boolean logic. To understand something complex, we need to understand the simpler constituents.

The Magic of Representation - Vector Space and Hyperplane

Most people are familiar with neural networks, cost functions, gradient descent, and backpropagation. However, beyond these building blocks is the magic of representations.

Features live in a multidimensional universe where the concept of a **hyperplane** classifies or clusters similar features together.

This idea applies equally to the simplest neural networks and to modern architectures such as Transformers.

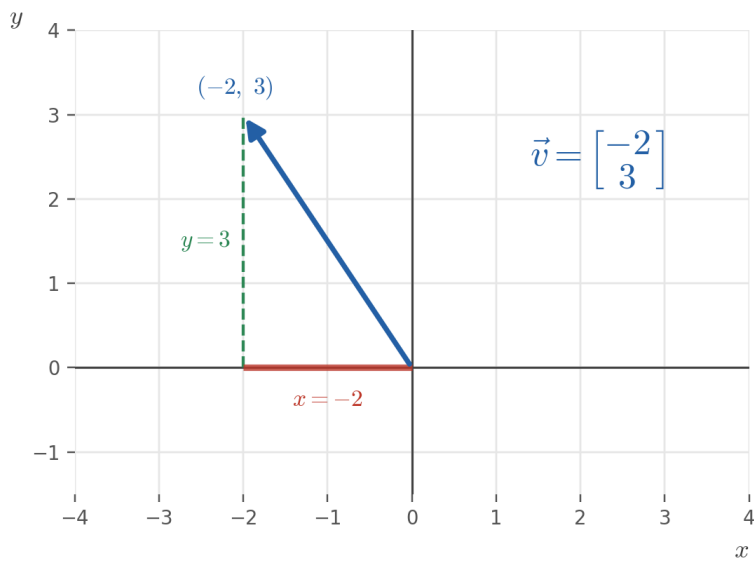
One of the earliest neural networks, **Rosenblatt's Perceptron**, introduced the idea of representing inputs as **vectors** and using the **dot product** to define a decision boundary — a hyperplane that separates input feature vectors.

First a short refresher.

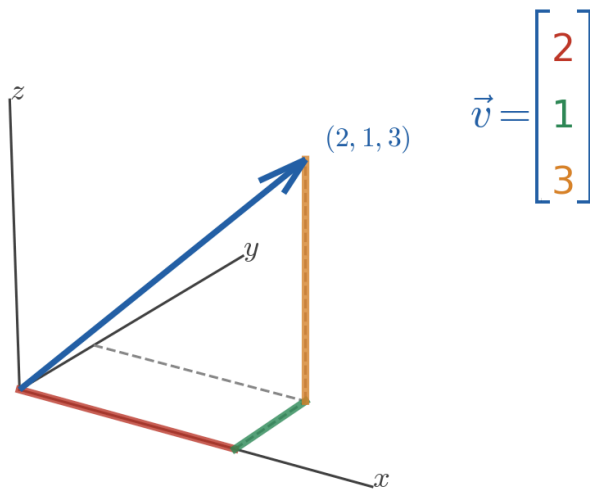
Vectors

A vector is an object that has both a magnitude and a direction. Example Force and Velocity. Both have magnitude as well as direction.

However we need to specify also a context where this vector lives -[Vector Space](#). For example when we are thinking about something like [Force vector](#), the context is usually 2D or 3D Euclidean world.



A vector in 2D: $\vec{v} = [-2, 3]^T$, drawn from the origin



A vector in 3D: $\vec{v} = [2, 1, 3]^T$, one component per axis

The easiest way to understand the Vector is in such a geometric context, say 2D or 3D cartesian coordinates, and then extrapolate it for other Vector spaces which we encounter but cannot really imagine.

Matrices - A way to represent Vectors (and Tensors)

Vectors are represented as matrices. A Vector is a one dimensional matrix. A matrix is defined to be a rectangular array of numbers. Example here is a [Euclidean Vector](#) in three-dimensional Euclidean space (or R^3) with some magnitude and direction (from (0,0,0) origin in this case).

A vector is written either as a column matrix (m1) or, after taking the transpose, as a row matrix (1m).

$$a = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}, \quad a^T = [a_1 \quad a_2 \quad a_3]$$

a_1, a_2, a_3 are the component scalars of the vector. A vector is represented as $\vec{a}$ in the **Vector notation** and as a_i in the **Index Notation**.

Two dimensional matrices can be thought of as one dimensional vectors stacked on top of each other.

This intuition is especially helpful when we use dot products on neural network weight matrices.

Dot Product

This is a very important concept in linear algebra and is used in many places in machine learning.

Algebraically, the dot product is the sum of the products of the corresponding entries of the two sequences of numbers.

if $\vec{a} = \langle a_1, a_2, a_3 \rangle$ and $\vec{b} = \langle b_1, b_2, b_3 \rangle$, then

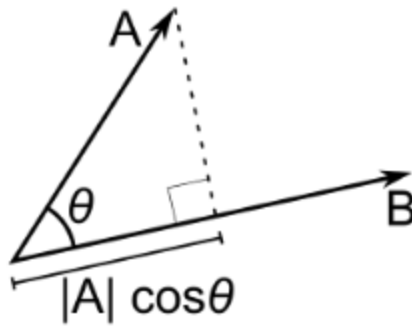
$$\vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + a_3 b_3 = a_i b_i \quad \text{in index notation}$$

In Matrix notation,

$$\vec{a} \cdot \vec{b} = [a_1 \quad a_2 \quad a_3] \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = a_i b_i$$

Geometrically, it is the product of the Euclidean magnitudes of the two vectors and the cosine of the angle between them

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos \theta$$



dotproduct

Note- These definitions are equivalent when using Cartesian coordinates (Ref [8](#), [9](#))

If two vectors point in roughly the same direction, their dot product is positive. If they point in opposite directions, the dot product is negative.

This simple geometric fact becomes a powerful computational tool.

Imagine a problem where we want to classify whether a leaf is healthy or diseased based on certain features. Each leaf is represented as a feature vector in a two-dimensional space (for simplicity).

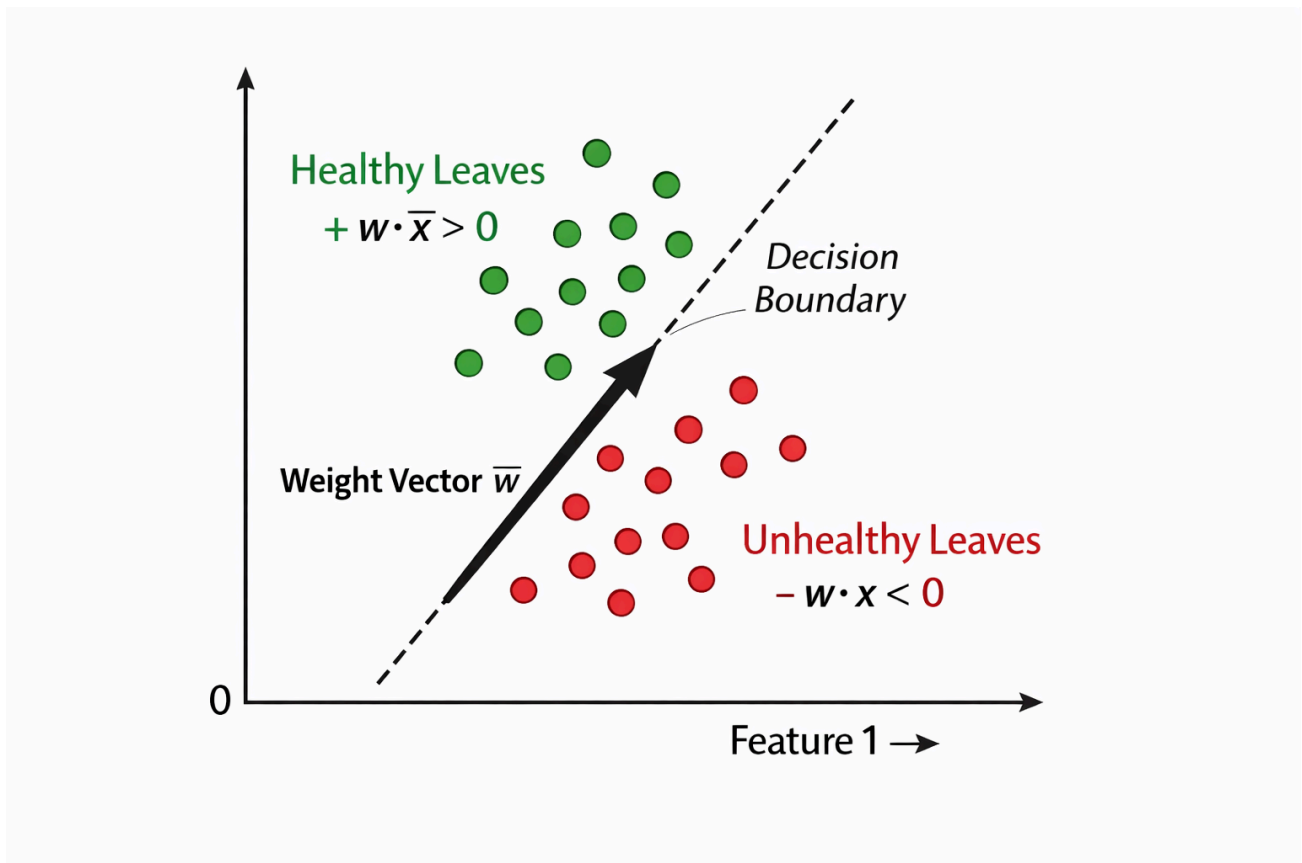
If we can find a weight vector such that:

Its dot product with healthy leaf vectors is positive

Its dot product with diseased leaf vectors is negative

then that weight vector defines a hyperplane that splits the feature space into two regions.

This is exactly how first artificial neuron-the Perceptron performs classification.

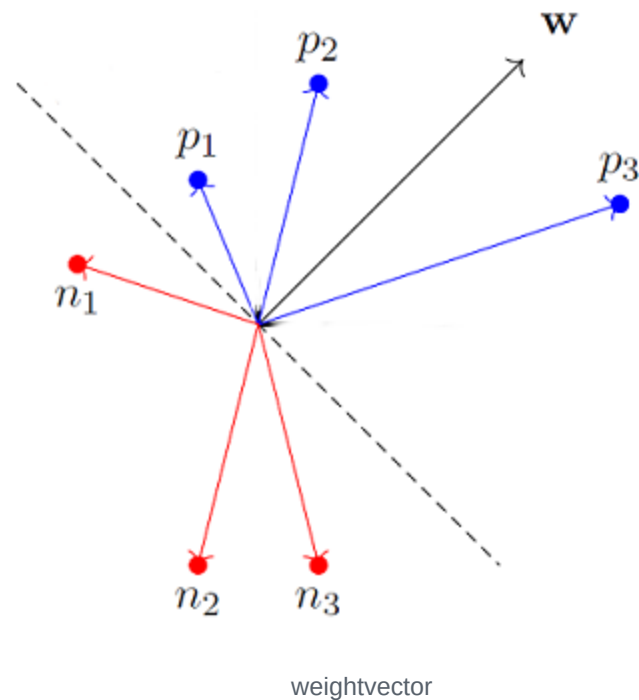


hyperplane1

Imagine we have a problem of classifying if a leaf is healthy or not based on certain features of the leaf. For each leaf we have some feature vector set assume it is a 2D vector space with say color as the feature for simplicity.

For any **input feature vector** in that vector space, if we have a **weight vector**, whose dot product with one feature vector of the set of input vectors of a certain class (say leaf is healthy) is positive, and with the other set is negative, then that weight vector is splitting the feature vector hyper-plane into two.

Or in a better way, which shows the vectors properly



In essence, we are using the weight vectors to split the hyper-plane into two distinctive sets.

For any new leaf, if we only extract the same features into a feature vector; we can *dot product* it with the *trained* weight vector and find out if it falls in healthy or diseased class.

Here is a Colab notebook to play around with this.[14](#)

Summary

What we have seen so far is that we can represent real world features as vectors residing in some N dimensional space.

We can then use the concept of hyperplane to split the feature space into two distinctive sets. This is the magic of Representation

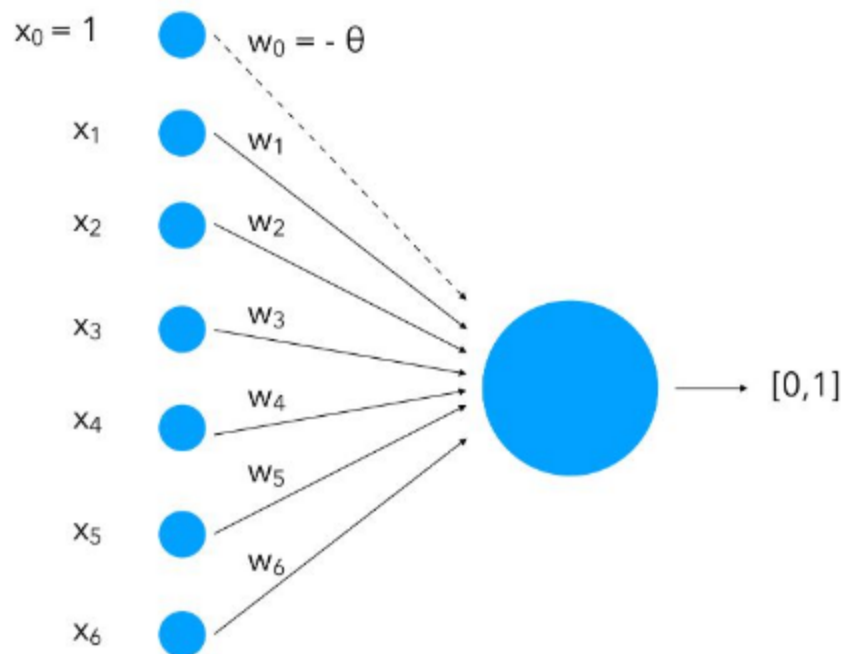
Next [Perceptron Training](#)

References

- [Vector Space - Wikipedia](#)
- [Introduction to Vectors - MathCentre](#)
- [Euclidean Vector - Wikipedia](#)

- [Dot Product - Paul's Online Math Notes](#)
- [Vector Math - MathOfNeuralNetworks](#)
- [Minsky's And-Or Theorem](#)
- [The Perceptron - Mael Fabien](#)
- [Rosenblatt's Perceptron Article](#)
- [Colab Notebook](#)
- [A Primer on Index Notation - John Crimaldi](#)

Perceptron Training via Feature Vectors & HyperPlane split



perceptron

Let's follow from the previous chapter of the Perceptron neural network.

We have seen how the concept of splitting the hyper-plane of feature set separates one type of feature vectors from other.

How are the weights learned?

You may have heard about **Gradient Descent**, which is the backbone of training modern neural networks. However, for the classic Perceptron, the learning algorithm is much simpler and relies on a geometric intuition.

The goal is to find a weight vector w that defines a hyperplane separating the two classes of data (e.g., Positive and Negative).

Note this term **hyperplane** is used in the context of feature vector space and is used throughout neural network learning.

The Intuition: Nudging the Vector

Imagine the weight vector w as a pointer. We want this pointer to be oriented such that: 1. It points generally in the same direction as **Positive** examples. 2. It points away from **Negative** examples.

We start with a random weight vector. Then, we iterate through our training data and check how the current w classifies each point.

- **If the classification is correct:** We do nothing. The weight vector is already doing its job for this point.
- **If the classification is wrong:** We need to “nudge” or rotate the weight vector to correct the error.

The Update Rules

Let's say we have an input vector x .

Case 1: False Negative The input x is a **Positive** example ($y = 1$), but our current w classified it as negative (dot product $w \cdot x < 0$). * **Action:** We need to rotate w *towards* x . * **Update:** $w_{new} = w_{old} + x$ * **Result:** Adding x to w makes the new vector more aligned with x , increasing the dot product for the next time.

Case 2: False Positive The input x is a **Negative** example ($y = 0$ or -1), but our current w classified it as positive (dot product $w \cdot x > 0$). * **Action:** We need to rotate w *away* from x . * **Update:** $w_{new} = w_{old} - x$ * **Result:** Subtracting x from w pushes it in the opposite direction, decreasing the dot product.

The Formal Algorithm

We can combine these rules into a single update equation. We often introduce a **learning rate** η (a small number like 0.1) to make the updates smoother, preventing the weight vector from jumping around too wildly.

For each training example (x, y_{target}) : 1. Compute prediction: $\hat{y} = \text{step_function}(w \cdot x)$ 2. Calculate error: $error = y_{target} - \hat{y}$ 3. Update weights:

$$w = w + \eta \cdot error \cdot x$$

This is known as the **Perceptron Learning Rule**.

$$\Delta w_j = \eta(y_{target} - \text{prediction})x_j$$

Note: This is distinct from Gradient Descent. Gradient Descent requires a differentiable activation function to compute gradients (slope). The Perceptron uses a “step function” (hard 0 or 1) which is

not differentiable. However, this simple rule is guaranteed to converge if the data is linearly separable.

A more rigorous explanation of the proof can be found in the book [Neural Networks by R.Rojas](#) or this [article](#).

Next: [Gradient Descent and Optimization](#)

References

- [Neural Networks - R. Rojas \(Chapter 4\)](#)
- [Perceptron Learning Algorithm - Towards Data Science](#)
- [Gradient Search - Alvarez](#)

You've finished the free sample

You now have the geometry: vectors, dot products, hyperplanes and how a perceptron learns. The rest of the book turns that intuition into the full mathematics of how modern networks train.

In the full book:

- **Chapter 3 - Gradient Descent:** a neural network as a chain of functions, with a worked numeric example and the chain rule step by step.
- **Chapter 4 - Backpropagation with Scalar Calculus:** why it is dynamic programming applied to the chain rule.
- **Chapter 5 - Backpropagation with Matrix Calculus:** gradient vectors, the Jacobian, VJP/JVP and the Hadamard product, summarised in NumPy.
- **Chapter 6 - Softmax and Cross-Entropy:** the full derivation of the famous $(p - y)$ gradient.
- **Chapter 7 - Neural Network Implementation:** a two-layer network in plain Python, trained from scratch, first with MSE and then with Softmax and Cross-Entropy.

Get the full book on Leanpub: leanpub.com/themathsofdeeplearning

Pay what you want from the minimum price, with free updates for life as new chapters and notebooks are added.

- Alex Punnen